

QA Copilot Tool Documentation

QA Copilot Documentation

What QA Copilot Is

QA Copilot is an AI-assisted QA platform for test engineers, QA leads, product teams, and admins. It helps teams turn requirements, designs, execution sheets, logs, URLs, and release notes into structured QA outputs such as test cases, automation drafts, bug analysis, release-risk assessments, website audits, and downloadable reports.

The platform combines:

- A Next.js web application for users and admins
- An Express API for business logic and AI workflows
- PostgreSQL with Prisma for users, projects, usage, subscriptions, and saved artifacts
- JWT authentication with admin approval
- OpenAI-powered analysis and generation
- Playwright-based website execution and visual QA

What the Tool Does

QA Copilot is designed to support the full QA lifecycle instead of only one testing task.

It helps teams:

- Generate test cases from requirement documents or pasted product content
- Convert manual test scenarios into starter automation scripts
- Analyze bugs from logs, stack traces, screenshots, or issue descriptions
- Generate realistic test data for positive, negative, and edge-case testing
- Turn execution notes into stakeholder-friendly QA reports
- Execute website test cases against a live URL and capture screenshot evidence
- Generate API test coverage from API specifications
- Assess release readiness and produce go, conditional go, or no-go recommendations
- Compare live website content with source documents or screenshots
- Compare live pages with design references for visual and responsive drift
- Run QA checks across multiple URLs for website-release validation
- Export outputs as JSON and, where applicable, PDF and evidence files

How It Works

1. User access

Users register through the web app. New users are created with PENDING approval status and

cannot log in until an admin approves the account.

2. Admin approval

An approved admin logs into the admin console and approves or rejects new users. Admins can also manage credits and review usage.

3. Dashboard and project workspace

Approved users log in to the dashboard, where they can:

- View credits and recent activity
- Create and select projects
- Run AI modules against a selected project
- Save generated outputs as project artifacts

4. AI and QA execution

Each module sends the user input to the backend API. Depending on the workflow, the backend:

- Parses uploaded files
- Sends structured prompts to OpenAI
- Runs Playwright for website execution or page capture
- Runs visual comparison logic for live-vs-reference audits
- Stores outputs and usage events in the database

5. Result storage and exports

Results are returned to the dashboard and can also be stored as project artifacts. Some workflows support downloading JSON, PDF, and screenshot evidence generated during execution.

Main User Flow

1. User registers

2. Admin approves the account

3. User logs in

4. User creates or selects a project

5. User runs one or more QA workflows

6. QA Copilot consumes credits, stores usage, and saves artifacts

7. User reviews results and exports files if needed

Roles and Permissions

User

Approved users can:

- Access the dashboard
- Create projects
- Run QA and AI workflows
- Generate and export outputs
- View recent activity and artifacts

Admin

Admins can do everything a user can, plus:

- Approve or reject new accounts
- Terminate access
- Assign extra credits
- Review user and plan data
- Inspect platform usage statistics
- Check OpenAI usage when admin usage credentials are configured

Core Functionalities Included

1. Test Case Generation

Purpose:

Convert PRDs, BRDs, user stories, specs, or uploaded documents into structured test cases.

Input:

- Pasted requirement text
- Uploaded requirement files

Output:

- Structured test scenarios
- Coverage-ready QA cases
- Saved project artifact

Typical use:

Create test coverage quickly from business or product documentation.

2. Website Test Execution

Purpose:

Run web-oriented test cases against a live site and capture evidence.

How it works:

- Parses executable cases from uploaded or pasted structured input
- Launches Playwright Chromium in headless mode

- Performs supported actions such as open, click, fill, select, submit, wait, and verify
- Captures screenshots for each executed case
- Returns pass, fail, or blocked status

Output:

- Execution summary
- Pass rate
- Go or no-go recommendation
- Screenshot evidence under public/execution-evidence

Typical use:

Run a lightweight automated website validation pass from structured manual test cases.

3. Automation Script Generation

Purpose:

Convert manual scenarios into starter automation code.

Supported frameworks:

- Playwright
- Cypress
- Selenium

Output:

- Framework-specific automation draft
- Saved automation artifact

Typical use:

Help QA or automation engineers move from manual validation to automation faster.

4. Bug Analysis

Purpose:

Analyze logs, errors, screenshots, or issue descriptions and suggest likely root causes.

Output:

- Root-cause summary
- Affected module
- Suggested fix direction

Typical use:

Speed up triage when developers or testers only have raw logs or limited evidence.

5. Test Data Generation

Purpose:

Create realistic sample data for testing.

Output:

- Positive records
- Invalid records
- Edge-case records

Typical use:

Generate QA-ready datasets for onboarding, payments, ecommerce, KYC, profile flows, and similar test areas.

6. Test Report Generation

Purpose:

Turn raw execution notes, CSV/XLSX outputs, or test results into a stakeholder-ready QA report.

The report can include:

- Summary
- Pass rate
- Go or no-go recommendation
- Critical issues
- Blockers
- Stakeholder actions
- Evidence requirements

Typical use:

Create cleaner release or QA summaries from raw execution sheets.

7. API Test Generation

Purpose:

Generate API test coverage from OpenAPI, Swagger, or pasted API specs.

Output:

- API test cases
- Example request payloads

Typical use:

Speed up API QA planning from documentation.

8. Release Risk Analysis

Purpose:

Turn QA findings into an executive-ready release decision.

The analysis can include:

- Readiness score
- Risk level
- Go, conditional go, or no-go recommendation
- High-risk modules
- Blockers
- Residual risks
- Rollback triggers
- Deployment guards
- Sign-off owners
- Monitoring plan
- Communication plan

Typical use:

Prepare a release-readiness brief for QA leads, product owners, and release managers.

9. Content Match

Purpose:

Compare a live page against a source document, content reference, or screenshot notes.

Checks can include:

- Missing content
- Content drift
- SEO mismatches
- Formatting inconsistencies
- Section and heading comparisons
- Visual comparison data
- Auto-generated bug-report suggestions

Typical use:

Validate whether published website content matches source material before or after release.

10. Design Match

Purpose:

Compare a live page against a design reference, design notes, or uploaded image.

Checks can include:

- Visual drift

- Responsive deviations
- Component-level differences
- Accessibility findings
- Design-token consistency
- Screenshot-to-code validation

Typical use:

Run design QA on implemented pages and detect fidelity issues.

11. Bulk URL QA

Purpose:

Review multiple URLs in one workflow for release or regression monitoring.

Output can include:

- Page-level findings
- Regression insights
- Workflow or routing issues
- CI/CD and monitoring suggestions
- Generated ticket suggestions

Typical use:

Run broader website QA checks across many pages during a release cycle.

12. Admin and Billing Operations

Included operational capabilities:

- Approve, reject, or terminate users
- Add credits to user accounts
- View platform-wide stats
- Review plan usage
- Trigger checkout flow for subscription changes
- Fall back to a local plan update when Stripe is not configured

Projects, Artifacts, and History

QA Copilot is project-based. Users can create projects and attach outputs to them.

Artifacts stored per project can include:

- Test cases
- Automation scripts
- Bug analyses

- Test data
- Test reports
- API tests
- Release-risk analyses
- Content-match results
- Design-match results
- Bulk URL QA results

This makes it easier to preserve QA history, evidence, and generated outputs in one place.

Credit-Based Usage

The tool uses credits for major workflows. Based on the current backend logic, the cost per action is:

- Generate test cases: 10
- Generate automation: 8
- Analyze bug: 5
- Generate test data: 4
- Generate test report: 6
- Execute test cases: 12
- Generate API tests: 9
- Analyze release risk: 7
- Content match: 12
- Design match: 14
- Bulk URL QA: 18

Each successful action:

- Deducts credits from the user balance
- Creates a usage event
- Can save an artifact against the selected project

Supported Inputs

Depending on the workflow, the platform supports:

- Plain text
- JSON
- CSV
- XLSX and XLS
- TXT
- Markdown

- PDF
- DOCX
- Images such as PNG, JPG, JPEG, and WEBP
- Website URLs

Outputs Generated by the Tool

QA Copilot can generate:

- Structured JSON responses
- Human-readable analysis summaries
- Automation code drafts
- Execution summaries
- Release-readiness assessments
- PDF reports
- Screenshot evidence
- Stored project artifacts

Technical Architecture

Frontend

- Next.js App Router
- React
- Tailwind CSS

Main UI areas:

- Landing page
- Registration and login
- User dashboard
- Project workspace
- Admin console

Backend

- Express API
- Route modules for auth, dashboard, projects, admin, AI, and billing

Important API groups:

- /api/auth
- /api/dashboard
- /api/projects
- /api/admin

- /api/ai
- /api/billing

Database

Prisma models currently manage:

- Users
- Subscription plans
- Subscriptions
- Projects
- Project artifacts
- Usage events

AI and Automation Layer

- OpenAI for generation and analysis
- Playwright Chromium for website execution and live-page capture
- Visual QA logic for screenshot comparison

Security and Access Model

The platform uses:

- JWT authentication
- Role-based authorization
- Admin approval before user login
- Protected API routes for approved users
- Admin-only routes for platform operations

Access rules:

- Public: landing, register, login, health
- Authenticated + approved: dashboard, projects, billing, AI workflows
- Authenticated + approved + admin: admin routes

Where the Tool Adds Value

QA Copilot is useful when teams need to:

- Reduce time spent converting documents into tests
- Speed up bug triage
- Produce stakeholder-ready QA outputs quickly
- Add lightweight website execution without building a large automation suite first
- Compare live websites against source content or designs

- Bring release-risk thinking into daily QA operations
- Keep QA artifacts organized by project

Ideal Use Cases

- QA engineers preparing coverage from product documents
- QA leads preparing release sign-off summaries
- Product teams validating website content before launch
- Design QA reviews for live pages
- Engineering teams needing fast automation starters
- Teams managing multiple website URLs during release windows

Important Notes and Limitations

- Most core workflows require an approved user account.
- AI features depend on valid OpenAI configuration.
- Billing can work in fallback mode if Stripe is not configured.
- Website execution works best with structured, browser-friendly steps.
- Visual matching is helpful for drift detection but should still be reviewed by a human for final acceptance.

Quick Summary

QA Copilot is a QA operations and AI-assistance platform that helps teams generate, execute, analyze, compare, and report on software and website quality work from one dashboard.

In simple terms, it helps users move from:

- Requirements to test cases
- Test cases to execution and automation
- Raw findings to bug analysis and reports
- Release evidence to go or no-go decisions
- Live pages to content and design validation